

实验5：变量译码器设计与应用（有点东西）

参考教材：Logic and Computer Design Fundamentals, 5th Edition (LCDF5)

第一部分：预备知识——理解译码器的数学基础

1.1 最小项 (Minterm)

这是理解译码器的**核心前提**，必须彻底掌握。

定义：对于 n 个变量，最小项是包含全部 n 个变量（每个变量以原变量或补变量的形式出现，且只出现一次）的**乘积项**。

以 3 变量 A、B、C 为例，共有 $2^3 = 8$ 个最小项：

编号	A	B	C	最小项表达式	符号
0	0	0	0	$A'B'C'$	m_0
1	0	0	1	$A'B'C$	m_1
2	0	1	0	$A'BC'$	m_2
3	0	1	1	$A'BC$	m_3
4	1	0	0	$AB'C'$	m_4
5	1	0	1	$AB'C$	m_5
6	1	1	0	ABC'	m_6
7	1	1	1	ABC	m_7

最小项的关键性质：

- 每个最小项在且仅在**一种**输入组合下值为 1
- 所有最小项之和 = 1（全集）
- 任意两个不同最小项之积 = 0（互斥）
- 任意布尔函数都能写成若干最小项之和（积之和标准式，SOP）

为什么这很重要：译码器的每个输出端 = 一个最小项，这是译码器能实现任意组合逻辑的根本原因。

1.2 标准积之和形式 (Canonical SOP)

任何组合逻辑函数 F 都可以写成：

$$F(A, B, C) = \sum_{i \in S} m_i$$

其中 S 是使 $F=1$ 的那些输入组合的编号集合。

例：某函数真值表如下：

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

$$\text{则 } F = \sum m(1, 3, 4, 6, 7) = m_1 + m_3 + m_4 + m_6 + m_7$$

$$\text{展开即：} F = A'B'C + A'BC + AB'C' + ABC' + ABC$$

这个性质是"译码器 + 门电路 = 任意组合函数"的数学依据。

第二部分：译码器的本质与定义

2.1 什么是译码器

直觉理解： 译码器就是一个"地址→位置"的转换器。

想象一个有 8 个房间的走廊，你给出一个 3 位的房间编号（000~111），译码器就把对应编号的那扇门打开（输出有效），其余所有门关闭。

形式化定义：

一个 n -to- 2^n 译码器有 n 条输入线和最多 2^n 条输出线。对于 n 条输入线的每一种二进制组合，至多有一条输出线被激活。

输入与输出的关系：

- 输入： n 位二进制编码 $(I_{n-1}, \dots, I_1, I_0)$

- 输出：第 k 条输出线 = 输入所表示的第 k 个最小项

$$\text{Output}_k = 1 \iff (I_{n-1} \dots I_1 I_0)_2 = k$$

2.2 译码与编码的对比（重要！）

特征	译码器 (Decoder)	编码器 (Encoder)
功能	二进制码 $\rightarrow$ 独热码 (one-hot)	独热码 $\rightarrow$ 二进制码
输入	n 位二进制	2^n 位（一次只有1位为1）
输出	2^n 位（一次只有1位有效）	n 位二进制
方向	压缩 $\rightarrow$ 展开	展开 $\rightarrow$ 压缩

独热码 (One-hot encoding)： 任意时刻只有一位为 1，其余全为 0。如 00100000 是 8 位独热码，表示位置 5。

第三部分：1-2 译码器（最简单案例，建立直觉）

3.1 结构分析

1 个输入变量 A ，2 个输出 D_0 、 D_1 。

真值表：

A	D0	D1
0	1	0
1	0	1

逻辑表达式：

$$D_0 = A' = m_0$$

$$D_1 = A = m_1$$

逻辑电路：

A	NOT	D0 (= A' = m0)
A		D1 (= A = m1)

观察： 两个输出正好是 1 变量的两个最小项。任意时刻恰好有一个为 1。

第四部分：2-4 译码器（深度推导）

4.1 问题建立

输入： A_1, A_0 (2 位)

输出： D_0, D_1, D_2, D_3 (4 位, 独热码)

4.2 完整真值表

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

4.3 逻辑表达式推导

方法：对每个输出，找出它为 1 的那行，写出对应最小项。

$$D_0 = A_1' \cdot A_0' \quad (A_1 = 0, A_0 = 0 \text{ 时为 } 1, \text{ 即 } m_0)$$

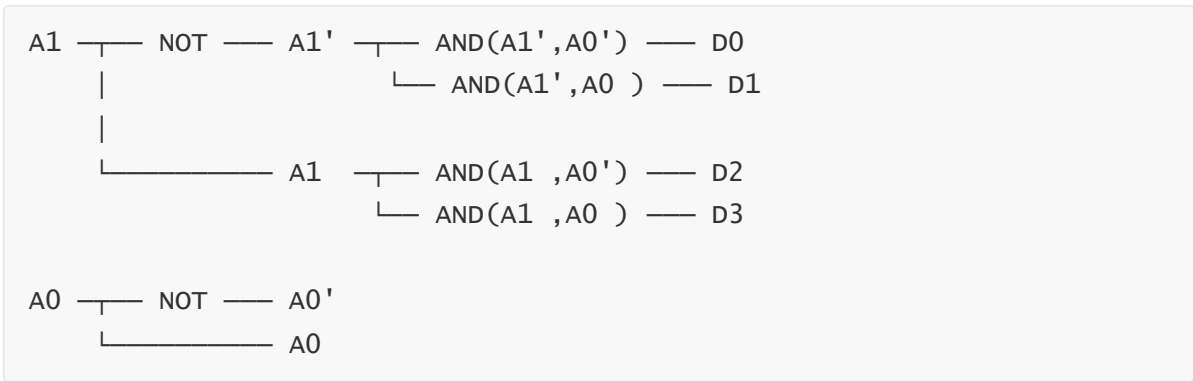
$$D_1 = A_1' \cdot A_0 \quad (A_1 = 0, A_0 = 1 \text{ 时为 } 1, \text{ 即 } m_1)$$

$$D_2 = A_1 \cdot A_0' \quad (A_1 = 1, A_0 = 0 \text{ 时为 } 1, \text{ 即 } m_2)$$

$$D_3 = A_1 \cdot A_0 \quad (A_1 = 1, A_0 = 1 \text{ 时为 } 1, \text{ 即 } m_3)$$

4.4 门级电路实现

需要的门：2 个 NOT 门 + 4 个 2 输入 AND 门 = 共 6 个门。



4.5 带使能端的 2-4 译码器

为什么需要使能端？

实际应用中，我们需要控制译码器是否工作。使能端（Enable，简称 EN 或 E）就是这个“总开关”。

加入使能端后的真值表（EN 高有效）：

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	×	×	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

更新后的逻辑表达式：

$$D_0 = EN \cdot A_1' \cdot A_0'$$

$$D_1 = EN \cdot A_1' \cdot A_0$$

$$D_2 = EN \cdot A_1 \cdot A_0'$$

$$D_3 = EN \cdot A_1 \cdot A_0$$

只需在每个 AND 门中增加 EN 输入（变为 3 输入 AND 门）即可。

第五部分：3-8 译码器（74HC138）——核心重点

5.1 器件概述

74HC138 是最常用的 3-8 译码器，理解它的每个细节都很重要。

引脚说明：

引脚名称	方向	功能
A, B, C	输入	地址选择端（A最低位，C最高位）
G1	输入	使能端，高电平有效
G2A'	输入	使能端，低电平有效（上划线/撇号表示取反）
G2B'	输入	使能端，低电平有效
Y0'~Y7'	输出	译码输出，低电平有效

注意命名习惯：上标 ' 或符号上划线表示低有效（active low）。

5.2 使能逻辑详细分析

74HC138 工作的条件：

$$\text{Enable} = G1 \cdot \overline{G2A'} \cdot \overline{G2B'} = G1 \cdot G2A \cdot G2B$$

即三个使能端**同时满足**：

- $G1 = 1$ （高电平）
- $G2A' = 0$ （低电平，即其内部取反后为1）
- $G2B' = 0$ （低电平）

为什么要设计成这种混合极性？

这是芯片设计者精心设计的，为级联扩展服务：

- $G1$ 接高位地址（高电平有效）
- $G2A'$ 、 $G2B'$ 接地或接低位控制（低电平有效）

实际使用中，常见接法：

- $G1$ 直接接 VCC（恒为1）或接高位地址
- $G2A'$ 和 $G2B'$ 直接接 GND（恒为0）或接高位地址的反

5.3 完整功能表

G1	G2A'	G2B'	C	B	A	Y0'	Y1'	Y2'	Y3'	Y4'	Y5'	Y6'	Y7'
0	×	×	×	×	×	1	1	1	1	1	1	1	1
×	1	×	×	×	×	1	1	1	1	1	1	1	1
×	×	1	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0

关键观察：

1. 使能无效时，所有输出为 **1**（高电平，即无效状态）
2. 使能有效时，被选中的输出为 **0**（低电平，即有效状态）
3. 任意时刻至多一个输出为低

5.4 74HC138 的逻辑表达式

设 $EN = G1 \cdot \overline{G2A'} \cdot \overline{G2B'}$ （使能信号）

由于输出低有效，每个输出是对应最小项取反：

$$Y'_0 = \overline{EN \cdot C'B'A'} = \overline{EN \cdot m_0}$$

$$Y'_1 = \overline{EN \cdot C'B'A} = \overline{EN \cdot m_1}$$

$$Y'_2 = \overline{EN \cdot C'BA'} = \overline{EN \cdot m_2}$$

⋮

$$Y'_7 = \overline{EN \cdot CBA} = \overline{EN \cdot m_7}$$

通用公式：

$$Y'_k = \overline{EN \cdot m_k}$$

内部实现： 当 $EN=1$ 时， $Y'_k = \overline{m_k} = m'_k$ ，即每个输出端是一个 NAND 门。（确实）

5.5 地址与变量的对应关系

74HC138 中：

- A = 最低有效位（LSB）
- B = 中间位
- C = 最高有效位（MSB）

对应的最小项下标计算：

$$k = C \times 4 + B \times 2 + A \times 1$$

例：C=1, B=0, A=1 $\rightarrow k = 4+0+1 = 5 \rightarrow Y'_5$ 有效

第六部分：使能端的深层作用——级联扩展

6.1 为什么需要级联

3-8 译码器只能处理 3 位地址，产生 8 个输出。如果需要处理更多位地址怎么办？答案是 **级联（Cascade）**。

核心思想： 用高位地址做片选（通过使能端），低位地址做本片译码。

6.2 两片 3-8 构成 4-16 译码器

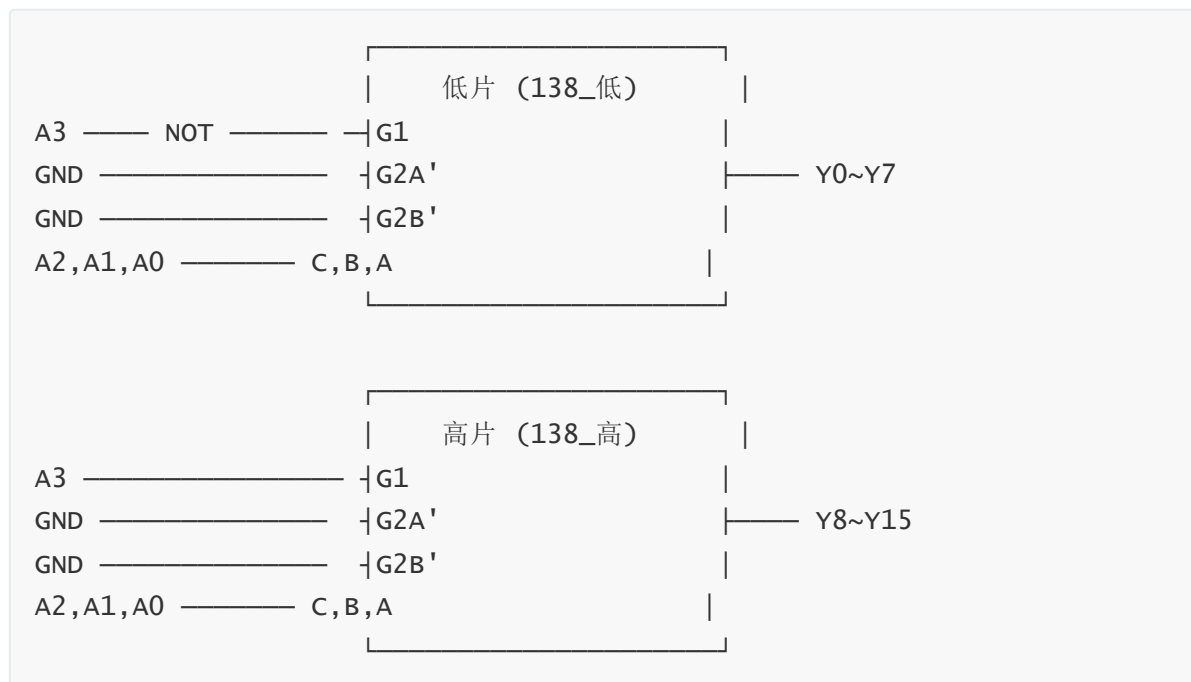
输入：4 位地址 $A_3A_2A_1A_0$

输出：16 个最小项 $Y_0 \sim Y_{15}$

分工：

- $A_3 = 0$ ：选择低片（产生 $Y_0 \sim Y_7$ ）
- $A_3 = 1$ ：选择高片（产生 $Y_8 \sim Y_{15}$ ）

连接方式：



详细分析：

当 $A_3 = 0$ 时：

- 低片：G1 = NOT(0) = 1 ✓, G2A'=0 ✓, G2B'=0 ✓ → **低片工作**
- 高片：G1 = 0 ✗ → **高片禁止**，所有输出为高（无效）

当 $A_3 = 1$ 时：

- 低片：G1 = NOT(1) = 0 ✗ → **低片禁止**
- 高片：G1 = 1 ✓, G2A'=0 ✓, G2B'=0 ✓ → **高片工作**

两片的低三位地址共用 A_2, A_1, A_0 ，所以：

- 当 $A_3 = 0$ 时，低片根据 $A_2A_1A_0$ 选择 Y0~Y7
- 当 $A_3 = 1$ 时，高片根据 $A_2A_1A_0$ 选择 Y8~Y15

输出编号：

- 低片 Yk 对应全局地址的第 k 个最小项 (k=0~7)
- 高片 Yk 对应全局地址的第 k+8 个最小项 (k=0~7, 全局为 8~15)

6.3 *三片 3-8 构成 5-32 译码器

用 5 位地址 $A_4A_3A_2A_1A_0$:

方案：先用一片 3-8（主译码器）对高位 A_4A_3 译码（只用前4个输出），再用这4个输出分别使能4组从属 3-8 译码器，每组处理低3位 $A_2A_1A_0$ 。

主译码器（138_master）：

输入： A_4, A_3 接 B, A （ C 接GND）， $G1=1$, $G2A'=0$, $G2B'=0$

输出： $Y0'\sim Y3'$ 分别控制4片从属译码器的使能

从属译码器 138_0（受 $Y0'$ 控制）：输出 $Y0\sim Y7$

从属译码器 138_1（受 $Y1'$ 控制）：输出 $Y8\sim Y15$

从属译码器 138_2（受 $Y2'$ 控制）：输出 $Y16\sim Y23$

从属译码器 138_3（受 $Y3'$ 控制）：输出 $Y24\sim Y31$

注意：主译码器 $Y0'\sim Y3'$ 是低有效，正好接从属译码器的 $G2A'$ （低有效使能端），完美对应！

第七部分：用译码器实现任意组合逻辑

7.1 理论基础（严格推导）

定理：任何 n 变量组合逻辑函数都可以用一个 $n\text{-}2^n$ 译码器和一个 OR 门（或等价地，NAND 门）实现。

证明：

设 $F(x_{n-1}, \dots, x_1, x_0)$ 是任意 n 变量布尔函数。

由布尔代数定理， F 可以写成标准积之和（SOP）形式：

$$F = \sum_{i \in S} m_i \quad \text{其中 } S = \{i \mid F = 1 \text{ 当输入为 } i\}$$

$n\text{-}2^n$ 译码器的第 k 个输出恰好是 m_k （高有效时）。因此：

$$F = \bigvee_{i \in S} \text{Output}_i$$

即将输出中属于 S 的那些端接到一个 OR 门，输出即为 F 。

若输出低有效（如74HC138），则：

$$Y'_k = \overline{m_k}$$

使用 NAND 门（De Morgan 定理： $A + B = \overline{\overline{A} \cdot \overline{B}}$ ）：

$$F = \sum_{i \in S} m_i = \overline{\prod_{i \in S} \overline{m_i}} = \overline{\prod_{i \in S} Y'_i} = \text{NAND}(Y'_{i_1}, Y'_{i_2}, \dots)$$

结论（记忆口诀）：

- 输出高有效 → 用 **OR** 门综合
- 输出低有效 (74HC138) → 用 **NAND** 门综合

7.2 多输出函数共用译码器的优势

传统方法： 每个输出函数需要单独化简，分别设计电路。

译码器方法： 多个输出函数共用同一个译码器，每个函数只需在输出端加一个 NAND 门。

优势：

1. **实现任意函数无需化简：** 不需要卡诺图化简，直接查真值表写最小项，接线即可
2. **多函数共用：** 一个译码器可以同时实现多个函数，大幅节省电路
3. **修改方便：** 改变函数只需修改后端的 NAND 门，译码器部分不变
4. **可编程性好：** 这实际上是 ROM（只读存储器）的工作原理

7.3 完整案例：用 74HC138 实现全加器

全加器真值表：

C_{in}	B	A	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

最小项提取：

$$Sum = \sum m(1, 2, 4, 7)$$

$$C_{out} = \sum m(3, 5, 6, 7)$$

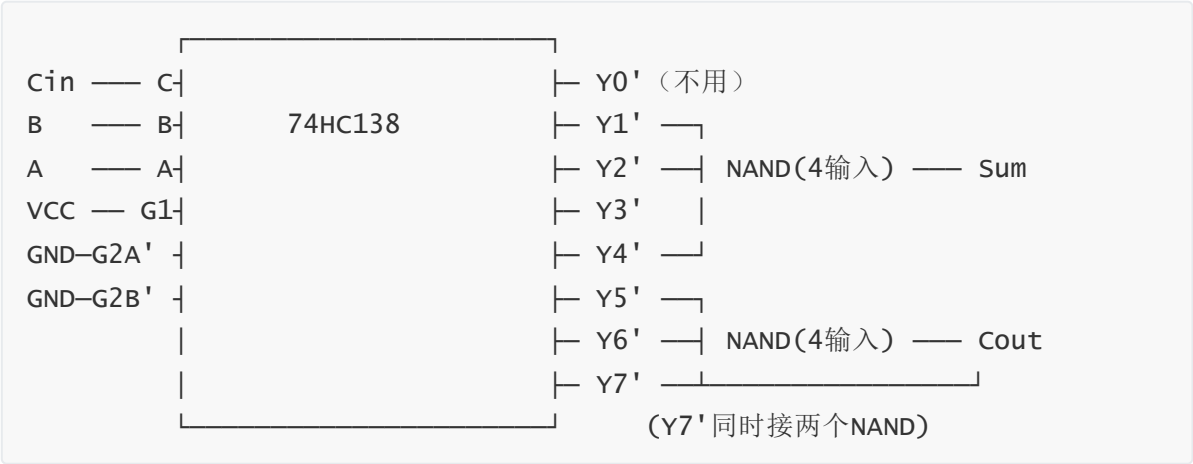
使用74HC138实现：

将 C_{in} , B, A 分别接 C, B, A (注意: C_{in} 为 MSB 接 C)

$$Sum = \text{NAND}(Y'_1, Y'_2, Y'_4, Y'_7)$$

$$C_{out} = \text{NAND}(Y'_3, Y'_5, Y'_6, Y'_7)$$

电路连接示意：



手动验证（取 Cin=0, B=1, A=1，即十进制3）：

- $k = C \times 4 + B \times 2 + A = 0 + 2 + 1 = 3$, $Y_3' = 0$, 其余=1
- $Sum = NAND(Y_1' = 1, Y_2' = 1, Y_4' = 1, Y_7' = 1) = NAND(1, 1, 1, 1) = 0 \checkmark$ （真值表 Sum=0）
- $Cout = NAND(Y_3' = 0, Y_5' = 1, Y_6' = 1, Y_7' = 1) = NAND(0, 1, 1, 1) = 1 \checkmark$ （真值表 Cout=1）

第八部分：LCDF5 对应章节详细映射

8.1 第六章知识体系

Chapter 6: Selected Design Topics	
├── 6-1: The Design Space	设计空间概念，从定制到通用
├── 6-2: Combinational Functional Blocks	← 实验5主要内容
│ ├── Value Fixing, Transferring, Inverting, Enabling	
│ ├── Decoder (译码器)	← 核心
│ ├── n-to-2^n Decoder 定义	
│ ├── 2-to-4 Decoder 设计	
│ ├── 3-to-8 Decoder (74HC138)	
│ ├── 使能端与片选	
│ ├── 级联扩展	
│ └── 实现组合函数	
│ ├── Encoder (编码器，对比学习)	
│ └── Multiplexer (多路选择器)	
├── 6-3: Rudimentary Logic Functions	基本逻辑功能（使能、传输等）
└── 6-4: Arithmetic Functions	加法器、减法等

8.2 关键定义 (LCDF5 原文对照)

Decoder (译码器) — LCDF5 Ch.6 原文:

"A decoder is a combinational circuit with n inputs and a maximum of 2^n outputs. The decoder decodes the n -input binary word to a maximum of 2^n distinct outputs, with only one output being a 1 at any time."

使能 (Enable) — LCDF5 将使能描述为基本逻辑功能之一:

"Enabling permits an input signal to pass through to an output. Disabling blocks an input signal from passing through to an output, replacing it with a fixed value."

第九部分：常见题型与解题模板

9.1 题型一：设计特定规模译码器

题：设计 4-16 译码器 (使用两片 74HC138)

步骤:

- 分析: 4位输入, 高位 A_3 做片选, 低3位 $A_2A_1A_0$ 做地址
- 低片 (产生 Y_0 - Y_7): $A_3 = 0$ 时工作
 - G1 接 A_3 经 NOT
 - G2A'、G2B' 接 GND
- 高片 (产生 Y_8 - Y_{15}): $A_3 = 1$ 时工作
 - G1 直接接 A_3
 - G2A'、G2B' 接 GND
- 两片的 C, B, A 都接 A_2, A_1, A_0

9.2 题型二：用译码器实现给定函数

题：用 74HC138 实现 $F(A, B, C) = \sum m(0, 2, 5, 7)$

步骤:

- 将 A,B,C 接译码器 (注意 C=MSB, A=LSB)
- G1=1 (接VCC), G2A'=0 (接GND), G2B'=0 (接GND)
- 写 NAND 表达式: $F = \overline{Y'_0 \cdot Y'_2 \cdot Y'_5 \cdot Y'_7}$
- 将 Y'_0, Y'_2, Y'_5, Y'_7 接一个 4 输入 NAND 门

验证 (取 A=1, B=0, C=0, 十进制1):

- $k = 0 \times 4 + 0 \times 2 + 1 = 1, Y'_1 = 0, \text{其余} = 1$

- $F = \text{NAND}(Y_0'=1, Y_2'=1, Y_5'=1, Y_7'=1) = 0$
- m_1 不在集合 $\{0,2,5,7\}$ 中, F 应为 0 ✓

9.3 题型三：分析给定译码器电路

分析步骤：

1. 确认输入接线：哪个变量接哪个地址位 (A/B/C)
2. 确认使能端：三个使能端是否满足工作条件
3. 看哪些输出线接入了后级门
4. 根据门的类型 (AND/OR/NAND/NOR) 写出布尔表达式
5. 化简验证

第十部分：深度理解——译码器与 ROM 的关系

10.1 可编程逻辑视角

"译码器 + 输出端的 NAND 阵列"正是 **ROM (只读存储器)** 的基本结构：

地址输入 → [译码器 (AND阵列)] → [OR/NAND阵列] → 数据输出

- **译码器部分**：固定的，将地址译码为唯一的行选信号（字线）
- **输出阵列部分**：可编程的，通过掩膜或熔丝决定哪些行参与每个输出（位线）

这说明 ROM 本质上就是一个"真值表存储器"，能存储并实现任意组合函数。

10.2 与多路选择器 (MUX) 的对比

特性	译码器+门	多路选择器 (MUX)
实现n变量函数	$n \cdot 2^n$ 译码器 + 门	$2^{n-1}:1$ MUX 或更小
多函数复用	一个译码器供多函数	每个函数需单独MUX
实现复杂度	$O(2^n)$ 输出线	$O(2^n)$ 数据输入
编程方式	修改后端门阵列	修改数据输入

附录：Verilog HDL 实现参考

74HC138 行为级建模

```
module decoder_74HC138 (  
    input  [2:0] A,          // 地址输入, A[0]=A, A[1]=B, A[2]=C  
    input      G1,          // 使能, 高有效  
    input      G2A_n,       // 使能, 低有效  
    input      G2B_n,       // 使能, 低有效  
    output reg [7:0] Y_n    // 输出, 低有效  
);  
  
    wire enable = G1 & (~G2A_n) & (~G2B_n);  
  
    always @(*) begin  
        Y_n = 8'hFF; // 默认全1 (全部无效)  
        if (enable)  
            Y_n = ~(8'h01 << A); // 对应位拉低  
    end  
endmodule
```

用译码器实现全加器

```
module full_adder_using_decoder (  
    input  Cin, B, A,  
    output Sum, Cout  
);  
  
    wire [7:0] Y_n;  
  
    // 实例化 74HC138, 使能端固定有效  
    decoder_74HC138 dec (  
        .A      ({Cin, B, A}), // C=Cin(MSB), B=B, A=A(LSB)  
        .G1      (1'b1),  
        .G2A_n   (1'b0),  
        .G2B_n   (1'b0),  
        .Y_n     (Y_n)  
    );  
  
    // Sum = m(1,2,4,7) → NAND(Y1', Y2', Y4', Y7')  
    assign Sum = ~(~Y_n[1] | ~Y_n[2] | ~Y_n[4] | ~Y_n[7]);  
    // 等价于: assign Sum = Y_n[1] & Y_n[2] & Y_n[4] & Y_n[7]; 然后取反  
    // 更直接: assign Sum = ~(&{Y_n[7], Y_n[4], Y_n[2], Y_n[1]});  
  
    // Cout = m(3,5,6,7) → NAND(Y3', Y5', Y6', Y7')  
    assign Cout = ~(&{Y_n[7], Y_n[6], Y_n[5], Y_n[3]});  
endmodule
```

总结：知识点层次结构

【译码器完整知识体系】

- 1. 数学基础
 - └─ 最小项 → 标准SOP → 任何函数都是最小项之和
- 2. 基本译码器
 - └─ 1-2：最简单，建立直觉（2个NOT/BUF）
 - └─ 2-4：掌握设计方法（2个NOT + 4个AND）
 - └─ 3-8（74HC138）：掌握全部细节
 - └─ 三个使能端极性（G1高有效，G2A'/G2B'低有效）
 - └─ 输出极性（低有效）
 - └─ 地址位顺序（A=LSB，C=MSB）
- 3. 级联扩展
 - └─ 核心思想：高位地址控制使能，低位地址做译码
 - └─ 2片3-8 → 4-16（最常考）
 - └─ 主从结构 → 任意扩展
- 4. 实现组合函数
 - └─ 高有效输出 → OR 门
 - └─ 低有效输出 → NAND 门（74HC138用这个）
 - └─ 多函数共用一个译码器
 - └─ 无需卡诺图化简，直接由真值表写最小项
- 5. 深层联系
 - └─ 译码器 + 输出阵列 = ROM 基本原理

常见错误总结

错误	原因	解决方法
输出全为高，从不变化	使能端极性接反	注意 G2A'/G2B' 低有效，接GND不是接VCC
实现的函数逻辑取反	混淆高/低有效	低有效输出用NAND而非OR
级联时片选错误	高位与使能连接错	A3控高片G1，经NOT控低片G1
地址位顺序错误	忽视A=LSB，C=MSB	赋值前确认 $k=C\times4+B\times2+A\times1$
Y7'接法遗漏	多函数共用同一输出未注意	一个输出端可接多个NAND门